



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
ENERO -MAYO 2026
CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:
ANÁLISIS AVANZADO Y DESARROLLO
TÍTULO ACTIVIDAD:
Actividad de evidencia 2_U2 Singleton

UNIDAD A EVALUAR:
Unidad 2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
Gutiérrez Cepeda Andrés - 23211009

NOMBRE DEL MAESTRO (A):
MARIBEL GUERRERO LUIS

5 de marzo 2026

Introducción

En el desarrollo de software, la gestión eficiente de la memoria y el control estricto de los recursos son fundamentales para construir aplicaciones robustas. El patrón de diseño creacional Singleton surge como una solución elegante para escenarios donde necesitamos garantizar que exista una, y solo una, instancia de una clase en todo nuestro sistema.

En esta actividad, aplique este concepto a un caso práctico: la simulación de una Central de Emergencias 911. En un entorno real, tener múltiples "centrales" descoordinadas generaría caos al despachar ambulancias o patrullas. A través del análisis de código en C#, el objetivo de esta práctica es desglosar la implementación del patrón Singleton mediante el uso de atributos estáticos y constructores privados. Asimismo, pondre a prueba la flexibilidad del sistema agregando nuevos operadores y simulando múltiples llamadas , para finalmente verificar, a nivel de memoria, que el sistema centralizado se mantiene como una instancia única.

Código

```
using System;

namespace SingletonApp
{
    public class Central_911
    {
        private static Central_911 _instance;
        private static readonly object _lock = new object();

        public string Central { get; private set; }

        private Central_911()
        {
            Central = "Central 911";
        }
    }
}
```

```

    }

    public static Central_911 Obtener_Instance()
    {
        if (_instance == null)
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Central_911();
                }
            }
        }
        return _instance;
    }

    public void ConectarLlamada(Operador operador, string tipoEmergencia)
    {
        Console.WriteLine("\nLlamada conectada con el operador " + operador.Nombre);
        operador.AtiendeEmergencia(tipoEmergencia);
    }
}

public class Operador
{
    public int Id_Operador { get; set; }
    public string Nombre { get; set; }

    public Operador(int id, string nombre)
    {
        Id_Operador = id;
        Nombre = nombre;
    }

    public void AtiendeEmergencia(string tipoEmergencia)
    {
        Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo:
{tipoEmergencia}");
        switch (tipoEmergencia)
        {
            case "Intento de suicidio":
                Console.WriteLine("Enviando unidades de apoyo y rescate.");
                break;
            case "Incendio":
                Console.WriteLine("Enviando bomberos.");
                break;
            case "Accidente":

```

```

        Console.WriteLine("Enviado paramedicos y oficiales.");
        break;
    case "Violeta":
        Console.WriteLine("Enviando una patrulla.");
        break;
    default:
        Console.WriteLine("Tipo de emergencia no reconocido.");
        break;
    }
}
}

public class Program
{
    public static void Main(string[] args)
    {
        // Instancias del Singleton
        Central_911 Llamada1 = Central_911.Obtener_Instance();
        Central_911 Llamada2 = Central_911.Obtener_Instance();

        // Operadores originales
        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");

        // AGREGADO: Más operadores
        Operador op3 = new Operador(3, "Marta");
        Operador op4 = new Operador(4, "Javier");

        // Llamadas originales
        Llamada1.ConectarLlamada(op1, "Incendio");
        Llamada2.ConectarLlamada(op2, "Violeta");
        Llamada1.ConectarLlamada(op1, "Accidente");
        Llamada2.ConectarLlamada(op2, "Intento de suicidio");

        // AGREGADO: Más llamadas
        Llamada1.ConectarLlamada(op3, "Violeta");
        Llamada2.ConectarLlamada(op4, "Incendio");
        Llamada2.ConectarLlamada(op3, "Accidente");

        // Verificando que es una única instancia
        Console.WriteLine("\nReferenceEquals: " + ReferenceEquals(Llamada1, Llamada2));
    }
}
}

```

```
private static Central_911 _instance;
```

1. El atributo estático de la instancia

Identificación en el código: `private static Central_911 _instance;`

Explicación: Este atributo es una variable declarada a nivel de clase. Tiene dos características fundamentales para el patrón Singleton:

Al ser `static`, la variable le pertenece a la clase `Central_911` en sí misma, y no a los objetos individuales que se pudieran crear. Esto permite que la variable comparta su valor (la instancia única) en toda la aplicación.

Al ser `private`, se asegura de que ninguna otra clase externa pueda acceder a ella o modificarla directamente, protegiendo así la integridad de la instancia única.

```
private Central_911()  
{  
    Central = "Central 911";  
}
```

2. El constructor

Identificación en el código: `private Central_911()`. Dentro de este, se inicializa la propiedad: `Central = "Central 911";`. Explicación: El constructor es el bloque de código responsable de inicializar un nuevo objeto cuando se crea. La clave absoluta de este patrón de diseño radica en su modificador de acceso: es `private`. Al ser privado, la clase "cierra sus puertas" y prohíbe explícitamente que cualquier otro archivo externo pueda invocar a este constructor.

¿Por qué no se pueden crear múltiples objetos?

No se pueden crear múltiples objetos de la clase `Central_911` porque su constructor es privado (`private Central_911()`)

Al tener un constructor privado, el lenguaje C# bloquea la posibilidad de usar la instrucción `new Central_911()` desde cualquier otra clase (como tu clase `Program`). La única manera de obtener un objeto de esta clase es a través del método público `Obtener_Instancia()`

Este método actúa como un guardián que controla la creación mediante la siguiente lógica:

Verifica si el atributo estático está vacío: `if (instance == null)`. (Nota: en tu código original dice `instance`, haciendo referencia a `_instance`).

Solo si está vacío (es decir, si es la primera vez que se pide), el método utiliza internamente el constructor privado para crear el objeto: `_instance = new Central_911();`

Finalmente, devuelve esa instancia: `return _instance;`

Si alguien vuelve a llamar al método, la condición `if (_instance == null)` será falsa, por lo que el método simplemente devolverá la instancia que ya había creado antes, garantizando matemáticamente que solo exista un objeto en memoria durante toda la ejecución.

Corrida:

```
Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Accidente
Enviado paramedicos y oficiales.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate.

Llamada conectada con el operador Marta
Operador Marta atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Javier
Operador Javier atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Marta
Operador Marta atendiendo emergencia de tipo: Accidente
Enviado paramedicos y oficiales.

ReferenceEquals: True
```

Conclusión

Realizar esta actividad representó un cambio importante en mi forma de entender la programación orientada a objetos, ya que normalmente estoy acostumbrado a utilizar la instrucción `new` libremente para crear tantos objetos como el sistema requiera. Sin embargo, aplicar el patrón Singleton a un caso práctico como la Central de Emergencias 911 me demostró el gran valor que tiene hacer exactamente lo opuesto: restringir y tener un control estricto sobre la memoria. Entender que cerrar las "puertas" de una clase mediante un constructor privado no es una limitación, sino una estrategia de diseño fundamental, fue uno de mis mayores aprendizajes; me quedó claro que en escenarios reales una mala gestión de instancias múltiples podría generar un caos total en la coordinación de recursos compartidos.

Por otro lado, la experiencia más satisfactoria de esta práctica fue poder comprobar la teoría directamente en la ejecución del código, especialmente al utilizar la instrucción `ReferenceEquals`. Ver que el compilador arrojaba un "True" confirmó matemáticamente que el sistema estaba perfectamente centralizado y que, a pesar de escalar el programa agregando más llamadas y nuevos operadores, todos trabajaban de manera sincronizada bajo la misma y única instancia. En definitiva, esta práctica consolidó mi comprensión sobre los patrones creacionales, dotándola de una perspectiva arquitectónica real que sin duda aplicaré en el futuro cuando necesite gestionar elementos críticos, como conexiones a bases de datos o configuraciones globales, de manera segura y eficiente.